Financial Pipeline Code Explanations

This document provides a line-by-line breakdown and structural trace of your financial data processing and analysis pipeline.

---

1. `load_statement(path)`

Purpose

This function acts as the initial data cleaning station. It reads a raw balance sheet or income statement CSV file, standardizes its table layout orientation, removes database prefixes (like `us-gaap_` or `f_`), handles empty rows, and sorts columns from the newest date to the oldest date.

Code

```
def load_statement(path):
    """Rows = line items, columns = dates (newest first). Auto-fixes orientation,
strips tag prefixes."""
    df = pd.read_csv(path, index_col=0)
    try:
        pd.to_datetime(df.columns)
    except Exception:
        df = df.T
    df.columns = pd.to_datetime(df.columns)
    df = df.apply(pd.to_numeric, errors="coerce")
    df = df[df.index.notna() & (df.index.astype(str).str.strip() != "")]
    df.index = [re.sub(r"^[A-Za-z]+(-[A-Za-z]+)?_", "", str(i)) for i in df.index]
    df = df[~df.index.duplicated(keep="first")]
    return df.sort_index(axis=1, ascending=False)
```

Line-by-Line Explanation

- **`def load_statement(path):`** Defines the function and accepts a string variable `path` representing the file location.
- **`df = pd.read_csv(path, index_col=0)`** Imports the CSV data into memory. `index_col=0` maps the very first text column as the table row labels (the index).
- **`try: pd.to_datetime(df.columns)`** Checks if the column headers can be parsed as valid dates. If they are already dates, this test passes and skips the `except` block.
- **`except Exception: df = df.T`** If the column headers are words (like "Cash") rather than dates, it triggers this block to transpose (flip sideways) the table layout, forcing the dates to become columns.
- **`df.columns = pd.to_datetime(df.columns)`** Overwrites the column headers with clean, standardized pandas `Datetime` objects for easy time filtering.
- **`df = df.apply(pd.to_numeric, errors="coerce")`** Converts text digits into float numbers. `errors="coerce"` safely changes empty slots or non-numeric words into `NaN` (blanks) without crashing.

- `df = df[df.index.notna() & (df.index.astype(str).str.strip() != "")]` Drops rows where the accounting line item name is entirely missing or consists only of blank white spaces.
- `df.index = [re.sub(r"^[A-Za-z]+(-[A-Za-z]+)?_", "", str(i)) for i in df.index]` Uses regular expressions to match and delete prefixes at the beginning of row headers (e.g., transforms `us-gaap_DebtCurrent` into `DebtCurrent`).
- `df = df[~df.index.duplicated(keep="first")]` Identifies rows with duplicate names after prefix removal and keeps only the first occurrence while dropping the rest.
- `return df.sort_index(axis=1, ascending=False)` Returns the DataFrame with columns sorted chronologically from the newest date to the oldest date (left to right).

---

2. `pick(df, key, notes, aliases=None)`

Purpose

This function is a fallback manager for line item names. Because companies use different naming conventions for the same accounting item (e.g., `"Inventory, Net"` vs `"Inventories"`), this function loops through alternative choices to piece together a unified data timeline, patching missing gaps from fallback rows where necessary.

Code

```
def pick(df, key, notes, aliases=None):
    """Coalesce per period: first alias with a value wins; fallbacks are
logged."""
    aliases = aliases or ALIASES
    result = None

    for alias in aliases[key]:
        if alias not in df.index:
            continue
        row = df.loc[alias]
        if result is None:
            result = row.copy()
        else:
            fill = result.isna() & row.notna()
            if fill.any():
                result[fill] = row[fill]
                notes.append(f"{key}: {int(fill.sum())} period(s) filled from
fallback row '{alias}'")
    if result is None:
        notes.append(f"{key}: not found in file")
    return result
```

Line-by-Line Explanation

- `aliases = aliases or ALIASES` If no custom alias dictionary mapping is provided, defaults to the global configuration map (`ALIASES`).

- `result = None` Prepares an empty variable to trace and build the finalized, blended timeline row for the target metric.
- `for alias in aliases[key]:` Loops down the ranked preference list of line-item names assigned to your specific key (e.g., checking `"inventory"` choices).
- `if alias not in df.index: continue` If the chosen alternative label is not present in the index of your spreadsheet, it skips forward to check the next alternative text name.
- `row = df.loc[alias]` Reaches into the spreadsheet and copies the timeline row belonging to that matched row name.
- `if result is None: result = row.copy()` If no primary choice data has been assigned yet, it clones this entire row as our baseline metric data stream.
- `else:` Triggers if a primary choice was already captured and the code is now looking at backup choices to patch holes.
- `fill = result.isna() & row.notna()` Creates a True/False mask mapping every column position where our primary row has a blank cell (`isna()`) but this fallback row contains an actual number (`notna()`).
- `if fill.any():` Passes true if there is at least one date period where the fallback row can successfully patch a gap in the primary series.
- `result[fill] = row[fill]` Injects values from the fallback row into the blank spaces of the primary series for those specific timeline slots.
- `notes.append(f"{key}: ... filled from fallback row '{alias}'")` Logs a confirmation tracking note specifying how many missing year dates were rescued by the backup row.
- `if result is None: notes.append(f"{key}: not found in file")` If the entire loop finishes without finding any matched rows from your list, it records a warning indicating the metric is entirely absent.
- `return result` Outputs the completed, compiled date series timeline array.

---

3. `build_items(bs, inc, aliases=None)`

Purpose

This is the main structural layout assembly line. It loops through checklists of standard balance sheet (`bs`) and income statement (`inc`) keys, extracts them using `pick()`, rotates the table orientation so that dates become rows and metrics become columns, and applies basic accounting logic to infer any missing totals.

Code

```
def build_items(bs, inc, aliases=None):
    notes, items = [], {}
    for k in BS_KEYS:
        s = pick(bs, k, notes, aliases)
        if s is not None:
            items[k] = s
    for k in IS_KEYS:
        s = pick(inc, k, notes, aliases)
        if s is not None:
            items[k] = s
    d = pd.DataFrame(items)
```

```
    d = d.sort_index(ascending=False)
    if "total_liabilities" not in d and {"total_assets", "equity"} <=
set(d.columns):
        d["total_liabilities"] = d.total_assets - d.equity
        notes.append("total_liabilities derived = total_assets - equity")
    if "gross_profit" not in d and {"revenue", "cogs"} <= set(d.columns):
        d["gross_profit"] = d.revenue - d.cogs
        notes.append("gross_profit derived = revenue - cogs")
    sd = d["short_debt"].fillna(0) if "short_debt" in d else 0
    ld = d["long_debt"].fillna(0) if "long_debt" in d else 0
    d["total_debt"] = sd + ld
    if "short_debt" not in d and "long_debt" not in d:
        notes.append("total_debt: no debt lines found, treated as 0")
    return d, notes
```

Line-by-Line Explanation

- `notes, items = [], {}` Initializes an empty log list (`notes`) and an storage workspace dictionary (`items`).
- `for k in BS_KEYS: s = pick(bs, k, notes, aliases)` Cycles through the Balance Sheet checklist keys, calling `pick()` to find their data strings.
- `if s is not None: items[k] = s` If data is found, it saves the timeline series under its standard key in our dictionary box.
- `for k in IS_KEYS: s = pick(inc, k, notes, aliases)` Repeats the exact same query retrieval loop for the Income Statement items checklist.
- `d = pd.DataFrame(items)` Combines all collected metric vectors into a single structured master spreadsheet. This step flips the layout so that **Dates become rows** and **Financial items become columns**.
- `d = d.sort_index(ascending=False)` Reorders the date row index from the newest period at the top down to the oldest period at the bottom.
- `if "total_liabilities" not in d and {"total_assets", "equity"} <= set(d.columns):` If total liabilities is missing but assets and equity exist, applies the basic balancing equation to derive it.
- `d["total_liabilities"] = d.total_assets - d.equity` Calculates total liabilities as $extTotalAssets - extTotalEquity$.
- `if "gross_profit" not in d and {"revenue", "cogs"} <= set(d.columns):` Derives gross profit if it is missing by checking if revenue and cost of goods sold are available.
- `d["gross_profit"] = d.revenue - d.cogs` Computes gross profit as $extRevenue - extCOGS$.
- `sd = d["short_debt"].fillna(0) if "short_debt" in d else 0` Isolates short-term debt and changes any isolated blank cells (`NaN`) to `0` to ensure safe addition.
- `ld = d["long_debt"].fillna(0) if "long_debt" in d else 0` Isolates long-term debt and changes any isolated blank cells (`NaN`) to `0`.
- `d["total_debt"] = sd + ld` Creates a brand new column named `"total_debt"` equal to the sum of short-term and long-term liabilities.
- `return d, notes` Returns the newly structured master table along with its operational calculation log.

---

4. `sdiv(a, b)` & `compute(d)`

Purpose

This is the math calculation engine. It takes the clean metrics matrix and applies standard formulas to generate key financial safety and growth metrics (liquidity ratios, solvency metrics, asset distribution shares, margins, and growth momentum velocity profiles).

Code

```
def sdiv(a, b):
    return a / b.where(b != 0)

def compute(d):
    r = pd.DataFrame(index=d.index)
    has = lambda *c: all(x in d for x in c)
    if has("current_assets", "current_liabilities"):
        r["current_ratio"] = sdiv(d.current_assets, d.current_liabilities)
        if has("inventory"):
            r["quick_ratio"] = sdiv(d.current_assets - d.inventory,
d.current_liabilities)
    if has("cash", "current_liabilities"):
        inv = d["short_investments"].fillna(0) if "short_investments" in d else 0
        r["cash_ratio"] = sdiv(d.cash + inv, d.current_liabilities)
    if has("equity"):
        r["debt_to_equity"] = sdiv(d.total_debt, d.equity)
    if has("total_assets"):
        if has("total_liabilities"):
            r["debt_ratio"] = sdiv(d.total_liabilities, d.total_assets)
        if has("equity"):
            r["equity_ratio"] = sdiv(d.equity, d.total_assets)
        if has("ppe"):
            r["ppe_share_of_assets"] = sdiv(d.ppe, d.total_assets)
        if has("inventory"):
            r["inventory_share_of_assets"] = sdiv(d.inventory, d.total_assets)
        if has("revenue"):
            r["asset_turnover"] = sdiv(d.revenue, d.total_assets)
    if has("retained_earnings", "equity"):
        r["retained_earnings_to_equity"] = sdiv(d.retained_earnings, d.equity)
    if has("operating_income", "interest_expense"):
        r["interest_coverage"] = sdiv(d.operating_income,
d.interest_expense.abs())
    if has("revenue"):
        if has("gross_profit"):
            r["gross_margin"] = sdiv(d.gross_profit, d.revenue)
        if has("operating_income"):
            r["operating_margin"] = sdiv(d.operating_income, d.revenue)
        if has("net_income"):
            r["net_margin"] = sdiv(d.net_income, d.revenue)
        r["revenue_growth"] =
d.revenue.sort_index().pct_change().sort_index(ascending=False)
    if has("net_income"):
        r["net_income_growth"] =
```

```
    d.net_income.sort_index().pct_change().sort_index(ascending=False)
    return r
```

Line-by-Line Explanation

- `def sdiv(a, b): return a / b.where(b != 0)` Safe division helper. If any denominator entry is exactly `0`, `.where()` converts it into a blank `NaN`. Dividing by `NaN` yields `NaN` safely instead of crashing your system with a zero division error.
- `r = pd.DataFrame(index=d.index)` Creates an empty results DataFrame `r`, duplicating the exact date row index structure to match timelines perfectly.
- `has = lambda *c: all(x in d for x in c)` A macro function that returns `True` only if every named column provided exists inside your current master table.
- `r["current_ratio"] = sdiv(d.current_assets, d.current_liabilities)` Computes the current ratio ($extCurrentAssets \div extCurrentLiabilities$) to check short-term debt coverage.
- `r["quick_ratio"] = sdiv(d.current_assets - d.inventory, d.current_liabilities)` Computes the quick ratio (Acid-Test), stripping away inventory since it takes longer to sell.
- `r["cash_ratio"] = sdiv(d.cash + inv, d.current_liabilities)` Measures conservative liquidity by dividing immediately available cash and marketable securities by current liabilities.
- `r["debt_to_equity"] = sdiv(d.total_debt, d.equity)` Divides calculated total debt obligations by shareholder equity to assess total financial risk leverage.
- `r["debt_ratio"] = sdiv(d.total_liabilities, d.total_assets)` Determines the percentage of company assets financed through debt obligations.
- `r["equity_ratio"] = sdiv(d.equity, d.total_assets)` Determines the percentage of assets financed directly by investor capital.
- `r["ppe_share_of_assets"] = sdiv(d.ppe, d.total_assets)` Measures what percentage of corporate wealth is locked inside fixed, physical infrastructure (Property, Plant, & Equipment).
- `r["inventory_share_of_assets"] = sdiv(d.inventory, d.total_assets)` Shows the portion of asset value currently sitting in warehouse product stocks.
- `r["asset_turnover"] = sdiv(d.revenue, d.total_assets)` Measures efficiency by tracking how many dollars of sales are generated per dollar of assets.
- `r["retained_earnings_to_equity"] = sdiv(d.retained_earnings, d.equity)` Evaluates what share of equity comes from accumulated internal profit rather than outside fundraising.
- `r["interest_coverage"] = sdiv(d.operating_income, d.interest_expense.abs())` Checks safety margins by dividing operating income by the absolute value (`.abs()`) of interest fees.
- `r["gross_margin"] = sdiv(d.gross_profit, d.revenue)` Calculates the core production markup efficiency score ($extGrossProfit \div extRevenue$).
- `r["operating_margin"] = sdiv(d.operating_income, d.revenue)` Computes operating profitability before taxes and financing costs.
- `r["net_margin"] = sdiv(d.net_income, d.revenue)` Tracks final bottom-line profitability percentage.
- `r["revenue_growth"] = d.revenue.sort_index().pct_change().sort_index(ascending=False)` Calculates Year-over-Year sales growth velocity. It temporarily sorts from oldest to newest (`.sort_index()`) to compute percentage changes, then flips it back to descending order.
- `r["net_income_growth"] = ...` Uses the same method to calculate profit growth velocity profiles over time.

5. `classify(name, value)`

Purpose

This is the automated diagnostic interpreter. It compares any computed numeric ratio value against fixed thresholds inside a benchmark rules dictionary to grade performance metrics as a `"strength"`, a `"weakness"`, or `"neutral"`.

Code

```
def classify(name, value):
    b = BENCH.get(name)
    if b is None or pd.isna(value):
        return None
    if b["hib"]:
        return "strength" if value >= b["strong"] else "weakness" if value <=
b["weak"] else "neutral"
    return "strength" if value <= b["strong"] else "weakness" if value >=
b["weak"] else "neutral"
```

Line-by-Line Explanation

- `b = BENCH.get(name)` Looks up the scoring guidelines assigned to the specific metric name inside the `BENCH` dictionary rules box.
- `if b is None or pd.isna(value): return None` Safety exit. If no rules exist for this metric, or if its calculated value is blank (`NaN`), it returns `None` and skips grading.
- `if b["hib"]:` Checks if **H**igher **I**s **B**etter (`hib` is True). Higher scores are preferred for items like liquidity or margins.
- `return "strength" if value >= b["strong"] else "weakness" if value <= b["weak"] else "neutral"` Under high-is-better logic: values matching or exceeding the strong ceiling are labeled a `"strength"`; values below the weak floor are labeled a `"weakness"`; everything else is marked `"neutral"`.
- `return "strength" if value <= b["strong"] else "weakness" if value >= b["weak"] else "neutral"` Under low-is-better logic (e.g., debt or leverage metrics): low values below the strong ceiling are classified as a `"strength"`, while high values exceeding risk limits are classified as a `"weakness"`.

6. `trend(series, name)`

Purpose

This function is a directional momentum tracker. It filters data across multiple years, isolates the four most recent data points, calculates year-over-year math differences, and pairs the direction with business health logic to return labels like `"rising_improving"` or `"falling_deteriorating"`.

Code

```
def trend(series, name):
    s = series.dropna()
    if len(s) < 3:
        return "insufficient_history"
    vals = pd.Series(s.sort_index().tail(4).values)
    diffs = vals.diff().dropna()
    direction = "rising" if (diffs > 0).all() else "falling" if (diffs < 0).all()
else "mixed"
    b = BENCH.get(name)
    if b is None or direction == "mixed":
        return direction
    improving = (direction == "rising") == b["hib"]
    return f"{direction}_{'improving' if improving else 'deteriorating'}"
```

Line-by-Line Explanation

- `s = series.dropna()` Cleans the metric timeline by stripping out any empty data cells.
- `if len(s) < 3: return "insufficient_history"` Drops out and logs an explicit warning tag if there are fewer than 3 years of data, as that is not enough to confirm a trend.
- `vals = pd.Series(s.sort_index().tail(4).values)` Reorders dates from oldest to newest, extracts the four most recent entries using `.tail(4)`, and drops text date strings to leave a clean numerical vector.
- `diffs = vals.diff().dropna()` Computes year-over-year mathematical shifts by subtracting each sequential row value from the next.
- `direction = "rising" if (diffs > 0).all() else "falling" if (diffs < 0).all() else "mixed"` Evaluates directional consistency. If every single change is positive, it returns `"rising"`. If every change is negative, it returns `"falling"`. If values fluctuated up and down, it returns `"mixed"`.
- `b = BENCH.get(name)` Extracts reference benchmarks from the rules map.
- `if b is None or direction == "mixed": return direction` If the metric is not in the rules map or has volatile, fluctuating directions, it returns the raw movement direction directly (e.g., `"mixed"`).
- `improving = (direction == "rising") == b["hib"]` Cross-references direction with corporate health rules. For example, a `"falling"` trajectory matched with a lower-is-better debt metric (`hib=False`) evaluates to `True` for `improving`.
- `return f"{direction}_{'improving' if improving else 'deteriorating'}"` Combines the findings into an automated text tag phrase.

---

7. `analyze(...)`

Purpose

This is the master orchestration manager. It coordinates the entire pipeline: loading raw files, calling internal assembly steps, applying audits, generating structured metric dashboards, categorizing scores, and saving everything to an external JSON report file.

Code

```
def analyze(bs_path, is_path, out_path="client_analysis.json", client="client",
            aliases=None, extra_notes=None, unreliable=None):
    unreliable = unreliable or {}
    bs, inc = load_statement(bs_path), load_statement(is_path)
    items, notes = build_items(bs, inc, aliases)
    notes = (extra_notes or []) + notes
    ratios = compute(items)
    latest = ratios.index[0]
    for c in ["current_assets", "current_liabilities", "total_assets", "equity",
"revenue"]:
        if c not in items:
            notes.append(f"WARNING: core item {c} missing - dependent metrics
skipped")

    metrics = {}
    for name in ratios.columns:
        valid = ratios[name].dropna()
        if valid.empty:
            notes.append(f"{name}: no data in any period")
            continue
        as_of, v = valid.index[0], valid.iloc[0]
        b = BENCH.get(name)
        band, flag = classify(name, v), None
        if as_of != latest:
            band = "stale"
            flag = f"latest data is from {as_of.strftime('%Y-%m-%d')}, not
{latest.strftime('%Y-%m-%d')}"
        if name in unreliable:
            band = "needs_verification"
            flag = f"data check failed: {unreliable[name]}"
        metrics[name] = {
            "as_of": as_of.strftime("%Y-%m-%d"),
            "latest_value": round(float(v), 4),
            "band": band,
            "flag": flag,
            "threshold": {"weak": b["weak"], "strong": b["strong"],
"higher_is_better": b["hib"],
                          "source": b["src"]} if b else None,
            "trend": trend(ratios[name], name),
            "history": {d.strftime("%Y-%m-%d"): (None if pd.isna(x) else
round(float(x), 4))
                        for d, x in ratios[name].items()},
        }

    pick_band = lambda label: [k for k, v in metrics.items() if v["band"] ==
label]
    result = {
        "client": client,
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "latest_period": latest.strftime("%Y-%m-%d"),
        "periods_available": [d.strftime("%Y-%m-%d") for d in ratios.index],
        "inputs_used_latest": {k: (None if pd.isna(v) else float(v)) for k, v in
```

```
    items.loc[latest].items()},
            "metrics": metrics,
            "summary": {
                "strengths": pick_band("strength"),
                "weaknesses": pick_band("weakness"),
                "neutral": pick_band("neutral"),
                "unbanded": [k for k, v in metrics.items() if v["band"] is None],
                "needs_verification": pick_band("needs_verification"),
                "stale": pick_band("stale"),
            },
            "data_notes": notes,
        }
        with open(out_path, "w") as f:
            json.dump(result, f, indent=2)
        return result
```

Line-by-Line Explanation

- `bs, inc = load_statement(bs_path), load_statement(is_path)` Triggers step 1 to clean up and structure both the incoming raw CSV data streams.
- `items, notes = build_items(bs, inc, aliases)` Triggers step 3 to run preference coalescing rules, rotate the table orientation, and merge data into a single master metrics table.
- `ratios = compute(items)` Runs step 4 to generate the mathematical ratios matrix across all matching historical periods.
- `latest = ratios.index[0]` Identifies the absolute newest calendar period row present in your dataset (e.g., `2025-12-31`).
- `for name in ratios.columns:` Begins looping through every calculated ratio column to generate detailed profile blocks.
- `as_of, v = valid.index[0], valid.iloc[0]` Finds the newest period containing actual numeric values (`as_of`) for this ratio, along with its calculation value (`v`).
- `band, flag = classify(name, v), None` Calls step 5 to assign a performance health score tag (`"strength"`, `"weakness"`, or `"neutral"`).
- `if as_of != latest:` Overrides the performance tag and changes the band to `"stale"` if the newest data available for this specific ratio is older than the latest period in the file.
- `if name in unreliable:` Overrides the performance tag and changes the band to `"needs_verification"` if this specific metric was manually flagged in the verification dictionary argument.
- `metrics[name] = { ... }` Compiles a detailed configuration profile for this ratio, including its latest rounded value, band group assignment, boundary limits, trend momentum tag, and complete historical trajectory logs.
- `pick_band = lambda label: [k for k, v in metrics.items() if v["band"] == label]` Creates a scanning macro shortcut that lists all metrics assigned to a target performance health band.
- `result = { ... }` Collects all components (client metadata, execution timestamp, input balances, ratio dashboards, categorized performance summary lists, and logs) into a single master dictionary structure.
- `json.dump(result, f, indent=2)` Formats and writes the master dictionary as clean, structured, readable text inside your permanent JSON output file.